# Advanced Browser Extension Security: Corpora, Artifacts, and Mechanisms for Threat Intelligence Research

The browser extension ecosystem represents one of the most complex and heavily targeted attack surfaces in modern computing. Functioning as highly privileged web applications distributed via centralized platforms such as the Chrome Web Store (CWS) and Mozilla Add-ons (AMO), extensions operate within a unique security boundary[1]. While modern web browsers rely on a multi-process architecture to isolate websites into unprivileged renderer processes, browser extensions bridge the gap between web content and the host operating system, routinely demanding expansive capabilities via application programming interfaces (APIs)[3]. These privileges allow extensions to manipulate the Document Object Model (DOM) across origins, intercept network traffic, access raw cookie stores, and execute arbitrary code[3].

The strategic importance of securing this ecosystem has been underscored by the explosive growth of Generative AI (GenAI) productivity tools. Driven by a 900% year-over-year increase in search interest for AI-related extensions, the broader AI browser market is projected to reach $76.8 billion by 2034[1]. This rapid expansion has attracted sophisticated threat actors who exploit the implicit trust users place in centralized web stores. To evade initial vetting, attackers frequently deploy "bait-and-switch" or supply-chain strategies: publishing a benign extension, cultivating a large user base, and subsequently pushing a malicious update after transferring ownership or compromising the developer's infrastructure[5]. Because browsers automatically update extensions in the background, millions of endpoints can be compromised simultaneously without any explicit user interaction[6].

To detect, analyze, and mitigate these threats, the security research community relies heavily on specialized datasets, Indicators of Compromise (IOC) feeds, and longitudinal version archives. This report provides an exhaustive, critical analysis of publicly available browser extension datasets, evaluates the mechanisms for obtaining consecutive extension version pairs for differential analysis, and unpacks the specific API abuses and evasion techniques identified across these corpora.

## Exhaustive Taxonomy of Publicly Available Extension Datasets

Threat intelligence operations and academic research groups have curated several critical datasets designed to track malicious extension campaigns. These repositories vary significantly in their scope, methodology, and the granularity of their metadata, ranging from automated scraping pipelines to deeply analyzed, human-verified threat feeds.

### Automated and Community-Maintained Threat Intelligence Repositories

For operational security teams, maintaining access to live, labeled datasets of malicious

extensions is paramount. Two prominent open-source repositories provide the foundation for most contemporary extension threat intelligence.

The malicious_extension_sentry repository, maintained under the GitHub handle toborrm9, serves as a highly active, automated database tracking extensions removed from the Chrome Web Store due to malware, security violations, or policy infractions[9]. This repository automates the collection and aggregation of extension removals from multiple sources, addressing the historical lack of a centralized, regularly updated public database[9]. The dataset tracks thousands of unique Chrome and Edge extension IDs, logging specific campaigns and tracking telemetry overlaps, such as cross-referencing over 6,200 unique IDs against specialized active probes[9]. Entries in this dataset are labeled strictly by their 32-character Extension ID, the extension's display name, and the date the item was added to the database, provided in both Markdown and Comma-Separated Values (CSV) formats[9]. The entire repository is published under the permissive MIT License and is accessible via its GitHub URL: https://github.com/toborrm9/malicious_extension_sentry[9]. A unique advantage of this dataset is its accompanying zero-install Python 3 scanner (malext.py), which allows researchers and administrators to perform instant, privacy-first local scans of Chromium-based browser profiles (including Chrome, Edge, Brave, Vivaldi, and Opera) against the live database[9].

Offering a more granular, campaign-oriented approach, the chrome-mal-ids repository (maintained by mallorybowes) functions as a community-maintained database specifically focused on Chrome and Edge IOCs[7]. This dataset diverges from simple scraping by offering deep threat modeling, classifying thousands of documented IOCs across dozens of specific campaigns, ranging from credential stealers and browser hijackers to supply chain compromises and ad fraud rings[7]. The labeling and metadata schema (current-list-meta.csv) is exceptionally detailed. It catalogs the EXTID, EXTID-NAME, DATE-DIS (the date the malicious behavior was first discovered), THREAT-TYPE (categorizations such as spyware, data-theft, or browser-hijack), BROWSER environment, STILL-ACTIVE (a binary flag indicating if it was live in the store at the time of reporting), and an OWNERSHIP-TRANSFER flag which is critical for identifying legitimate extensions that were acquired and weaponized by threat actors[7]. Furthermore, the dataset distinguishes between independently human-verified entries and automated delta imports via a TPCI-VERIFY status field[7]. Licensed under the Creative Commons Attribution 4.0 International (CC BY 4.0) license, the dataset is hosted at https://github.com/mallorybowes/chrome-mal-ids[7]. This resource is explicitly engineered for enterprise defense integration, offering STIX 2.1 bundles, MISP event JSON exports for manual import, and Sigma detection rules, allowing SIEM systems to automatically poll the repository for fresh IOCs[7].

## Academic Corpora and Behavioral Datasets

Beyond live threat feeds, academic institutions have produced massive foundational datasets derived from the static and dynamic analysis of the broader extension ecosystem. These corpora are essential for training machine learning classifiers and understanding macroeconomic trends in extension development.

The "You've Changed" dataset, introduced in the seminal 2020 ACM SIGSAC paper *You've*

*Changed: Detecting Malicious Browser Extensions through their Update Deltas* by Pantelaios et al., represents a critical historical archive focused entirely on the evolutionary trajectory of extensions[5]. The researchers curated an initial dataset of 922,684 different extension versions across 215,941 unique extensions collected over a six-year period[5]. Through a hybrid methodology combining Natural Language Processing (NLP) anomaly detection on user reviews with Abstract Syntax Tree (AST) code delta clustering, the researchers labeled 143 specifically malicious extensions, grouping them into 21 distinct malicious clusters[6]. The labeling in this dataset uniquely focuses on the precise API abuse sequences introduced during a benign-to-malicious update[6]. The dataset and its associated analytical frameworks are generally open-sourced under academic licenses on GitHub, frequently mirrored and extended by subsequent researchers investigating differential static analysis[13].

Focusing on the dynamic execution of extensions, the dataset generated by the USENIX Security 2014 paper *Hulk: Eliciting Malicious Behavior in Browser Extensions* remains a cornerstone of behavioral analysis[16]. This corpus comprises dynamic analysis traces for 48,332 Chrome extensions, tested rigorously against over one million URLs[16]. The dataset labels broad classes of malicious behavior, including affiliate fraud, credential theft, and ad injection. A defining feature of this dataset is its use of "HoneyPages"—dynamic, synthetically generated web pages designed to adapt structurally to an extension's expectations, forcing dormant malicious event handlers to execute their payloads in a monitored environment[16]. While the raw dynamic execution traces are primarily accessible via academic requests to the original authors, the structural methodologies pioneered in the Hulk dataset have deeply influenced subsequent behavioral corpora[17].

Subsequent large-scale studies have further expanded the academic data pool. The *SimExt* dataset, utilized for evaluating the effectiveness of Chrome Web Store vetting processes, compiled 366,617 extensions downloaded over four years[19]. This dataset is uniquely labeled with vetting statuses assigned by Google moderators, identifying "infringing clusters" of highly similar extensions where at least one member was removed for policy violations, thereby allowing researchers to measure the survival rate and impact of republished malicious clones[19].

Additionally, recent studies focusing on the GenAI threat landscape have curated targeted datasets. By actively monitoring AI-related keywords, researchers collected 5,551 unique Chrome extensions released in 2025 alone, eventually isolating a ground-truth dataset of 341 known malicious extensions (29 of which were specifically GenAI-themed) through rigorous static and network analysis[1].

## Analytical Frameworks Generating Local Corpora

While not centralized datasets in themselves, specific open-source frameworks are instrumental in generating custom, highly targeted datasets for local analysis.

The ExtAnalysis tool, developed by Tuhinshubhra, is a robust local static analysis framework for Chrome and Firefox extensions. Licensed under GPL-3.0 and available at https://github.com/Tuhinshubhra/ExtAnalysis, the tool processes .crx and .xpi archives to automatically extract URLs, domains, IPv6 and IPv4 addresses, Bitcoin addresses, and Base64 encoded strings[21]. By passing known malicious extensions through this framework, researchers

can rapidly generate bespoke IOC datasets for localized threat hunting.
Similarly, the chrrel/extensions repository provides a specialized web scanning infrastructure based on the Chrome DevTools Protocol. This tool generates datasets by analyzing interactions between websites and browser extensions, specifically targeting the abuse of Web Accessible Resources (WARs) and message-passing APIs like window.postMessage and runtime.sendMessage[22]. This infrastructure is vital for identifying instances where websites attempt to fingerprint installed extensions or where extensions improperly expose privileged APIs to unauthenticated web origins[22].

| Dataset / Tool Name | Scope and Primary Focus | Size and Volume | Labelling and Metadata | License | Public URL / Location |
|---|---|---|---|---|---|
| **malicious_extension_sentry** | Automated IOC aggregation, takedown tracking | Thousands of unique IDs; actively tracks 6.2k+ | 32-char EXTID, Name, Date Added | MIT | github.com/toborrm9/malicious_extension_sentry |
| **chrome-mal-ids** | Community-maintained TI, Campaign attribution | Dozens of campaigns, thousands of IOCs | EXTID, Threat-Type, Ownership-Transfer, Verification Status | CC BY 4.0 | github.com/mallorybowes/chrome-mal-ids |
| **"You've Changed" Corpus** | Code update deltas, benign-to-malicious transitions | 922,684 versions across 215,941 extensions | API abuse sequences, NLP anomaly scores, AST deltas | Academic / OSI | GitHub (Author Repositories) |
| **Hulk (HoneyPages)** | Dynamic behavioral execution, DOM manipulation | 48,332 extensions driven by 1M+ URLs | Behavioral classes (fraud, theft, injection) | Academic | Academic Request / Mirrors |

| | | | | | |
|---|---|---|---|---|---|
| **SimExt Vetting Dataset** | Extension similarity, vetting evasion analysis | 366,617 extensions collected over 4 years | Vetting labels, Infringing Clusters, User counts | Academic | Academic Publication Supplements |
| **GenAI Malware Corpus** | Exfiltration and evasion in AI extensions | 5,551 targeted extensions; 341 confirmed malicious | AI themes, obfuscation flags, C2 communication | CC BY 4.0 | Academic Publication Supplements |
| **ExtAnalysis** | Static analysis framework for local extraction | Target-dependent (generates local datasets) | Extracted URLs, IPs, Crypto addresses, Base64 strings | GPL-3.0 | github.com/Tuhinshubhra/ExtAnalysis |

# Mechanisms and Vectors for Acquiring Consecutive Extension Versions

Analyzing a static, single version of an extension frequently fails to capture the true intent of the developer. Threat actors routinely employ sophisticated evasion techniques, heavy code obfuscation, and delayed execution timers designed explicitly to bypass the initial, automated vetting processes of the extension stores[20]. The most definitive methodology for identifying supply chain attacks, compromised developer accounts, and stealthy malware injections is differential analysis. This technique requires comparing two consecutive versions of the exact same extension—the older, benign version against the newer, malicious version—to isolate the precise code delta responsible for the threat[5].

Acquiring these consecutive versions is a significant technical challenge. Official browser extension stores are architected to serve only the latest version of an application, deliberately discarding historical builds to force users onto patched, secure software. Consequently, researchers must rely on a combination of first-party protocols, third-party archives, and developer ecosystems to manually construct longitudinal version histories. The source of the version pairs directly dictates whether the resulting dataset yields benign evolutionary changes or captures the critical moment of malicious compromise.

## First-Party Telemetry: Browser Update Endpoints

The most authoritative source for acquiring extension binaries (packaged as .crx files for

Chrome or .xpi files for Firefox) is the official update endpoint utilized by the browsers themselves.

Google Chrome manages its background extension updates using the Omaha protocol, an XML-based communication standard. When Chrome checks for updates, it initiates an HTTPS POST request to the update endpoint, overwhelmingly routed to https://clients2.google.com/service/update2/crx[24]. To acquire consecutive versions directly from this endpoint, researchers must implement longitudinal polling. By scripting periodic requests to the Omaha endpoint and supplying a specific extension ID alongside a spoofed, artificially lowered version number within the XML payload, the endpoint interprets the client as outdated and responds with a direct download URL for the latest .crx file[24].

Similarly, Mozilla Firefox employs continuous querying against endpoints such as https://versioncheck.addons.mozilla.org/ and https://services.addons.mozilla.org/api/v3/addons/... to determine if a newer .xpi package is available for installed add-ons[2].

Continuous polling of these first-party endpoints primarily yields **benign pairs**. Because researchers must collect this data proactively over an extended period, they are fundamentally archiving normal, benign software development cycles. However, the immense value of this methodology is realized when an extension within the monitored cohort is eventually compromised, sold, or weaponized. Because the researcher possesses the full, cryptographically verified history of the extension directly from Google or Mozilla, they capture the exact benign-to-malicious transition in real-time. This proactive, continuous scraping is the foundational data collection mechanism required to build datasets like the "You've Changed" corpus[5].

## Third-Party Archival and Analytics Services

Because the retroactive collection of older versions from official stores is impossible once an update is globally pushed, third-party archival services fill a critical gap in threat intelligence by independently downloading and hosting historical .crx files.

Crx4Chrome operates as a widely utilized third-party repository that explicitly archives and hosts historical versions of Chrome extensions[28]. For any given extension in its database, Crx4Chrome provides direct download links to a sequence of consecutive historical builds. For instance, a researcher can effortlessly download version 0.75.0 (Updated: May 20, 2024), version 0.74.1 (Updated: June 21, 2023), and version 0.74.0 (Updated: June 1, 2023) of the same extension from a single page[28]. The platform ensures data integrity by publishing critical cryptographic metadata for each archive, including the precise File Size, MD5, SHA1, and SHA256 hashes[28]. Searching through Crx4Chrome yields **primarily benign pairs**, as the service focuses on archiving popular, highly-ranked extensions with stable user bases. However, if an extension is archived by the platform just prior to being flagged and removed from the official Chrome Web Store, Crx4Chrome may inadvertently preserve the malicious update, providing researchers with a rare, retroactive window into a bait-and-switch attack without requiring a pre-existing polling infrastructure.

Chrome-Stats serves as an advanced analytics platform tracking the performance, metadata, and version history of extensions[31]. Beyond facilitating the download of current and historical

.crx files via API access or custom helper tools[32], Chrome-Stats provides an invaluable feature for threat hunters: the tracking of permission deltas[4]. The platform visualizes the exact dates when an extension requested expanded privileges—such as the sudden addition of webRequest or cookies permissions—between version updates[4]. These metadata deltas serve as high-fidelity leading indicators that a previously benign extension is preparing for data exfiltration, allowing analysts to target specific version transitions for deeper reverse engineering.

## Developer Ecosystems: Open-Source Releases

For extensions developed entirely as open-source projects, platforms like GitHub serve as pristine repositories for obtaining consecutive versions. Developers frequently utilize GitHub Releases to publish the source code associated with specific semantic version tags. Retrieving data from GitHub provides researchers with the full, uncompiled source code, entirely bypassing the heavy minification and obfuscation inherently present in the packed .crx files distributed on the Web Store[20].

Data acquired from GitHub Releases yields **exclusively benign pairs**. Threat actors executing supply chain attacks, even against open-source projects, do not commit their malicious payloads to public repositories. Instead, they inject the heavily obfuscated malicious code directly into the compiled .crx payload uploaded to the Google Developer Dashboard. Consequently, GitHub releases are best utilized by researchers to train baseline machine learning models on what normal, benign code evolution looks like. By establishing a baseline of standard AST modifications during routine feature updates, security systems can more accurately detect the anomalous code structures introduced during a malicious, off-repository update.

## Academic Corpora Specializing in Deltas

Datasets specifically constructed around the concept of update deltas, such as the "You've Changed" corpus, represent the synthesis of these acquisition methods. By continuously scraping the CWS and correlating precise version release timestamps with Sudden spikes in negative user reviews containing keywords like "SPYWARE" or "VIRUS," researchers successfully isolated the exact transition points[6]. This complex methodology specifically guarantees the delivery of **benign-to-malicious pairs**. By comparing the AST of the last known benign version against the flagged malicious version, the dataset isolates the exact JavaScript API abuse sequence, effectively generating a signature for the malicious update[6].

| Source for Version Pairs | Acquisition Mechanism | Yield Type | Primary Use Case |
|---|---|---|---|
| **Chrome Omaha Endpoint** | Longitudinal polling via XML, spoofing older version numbers | **Benign Pairs** (captures malicious transition if monitored over time) | Building proprietary, cryptographically verifiable historical archives. |

| | | | |
|---|---|---|---|
| **Mozilla AMO Endpoints** | Continuous querying of versioncheck APIs | **Benign Pairs** (captures malicious transition if monitored over time) | Tracking organic development cycles of Firefox add-ons. |
| **Crx4Chrome** | Web-based third-party historical archive | **Primarily Benign** (rarely retroactive malicious preservation) | Retroactive retrieval of older versions without pre-existing polling. |
| **Chrome-Stats** | Analytics API, manifest and metadata tracking | **Primarily Benign** (excellent for metadata) | Tracking permission deltas as leading indicators of compromise. |
| **GitHub Releases** | Direct download of tagged source code commits | **Exclusively Benign** | Training ML baselines on normal AST code evolution; bypasses minification. |
| **Academic Corpora (e.g., "You've Changed")** | Pre-processed correlation of CWS scraping and NLP anomaly detection | **Benign-to-Malicious Pairs** | Analyzing the exact API abuse sequences introduced during weaponization. |

# Anatomy of Extension Malware: Evasion, Abuse, and the Manifest V3 Paradigm Shift

The analysis of consecutive version pairs through the aforementioned datasets consistently reveals specific, recurring patterns of privilege escalation, data exfiltration, and evasion employed by threat actors within the browser environment. Modern browsers rely on a multi-process design, separating the highly privileged browser process from the sandboxed renderer processes[3]. Extensions, however, are granted permissions that effectively pierce these sandboxes.

## Privilege Escalation and Data Exfiltration

When an extension transitions from benign to malicious, differential analysis typically highlights

the introduction of highly sensitive API calls designed to access the DOM, intercept network requests, and exfiltrate cookie stores.

Extensions that request the cookies permission gain the ability to query the entire browser cookie store across all domains, completely bypassing the fundamental security mechanism of the Same-Origin Policy (SOP). Malicious updates routinely introduce background scripts that iterate through session cookies for high-value targets—such as corporate Single Sign-On (SSO) portals, banking institutions, and social media platforms—and silently exfiltrate them to a Command and Control (C2) server. This provides the attacker with persistent access to authenticated sessions without ever requiring the user's explicit credentials[3].

Historically, the interception of network traffic was facilitated by the powerful webRequest and webRequestBlocking APIs. These interfaces allowed extensions to pause, inspect, and modify outbound HTTP and HTTPS headers in real-time[3]. Code deltas from compromised extensions often reveal the sudden implementation of these APIs to silently alter affiliate links within e-commerce traffic, inject unauthorized advertising payloads into the user's web view, or downgrade secure connections[16].

Furthermore, threat actors heavily abuse content scripts to manipulate the Document Object Model (DOM). By injecting arbitrary JavaScript directly into the execution context of viewed webpages, extensions can overlay invisible phishing forms over legitimate login portals, scrape rendered text to steal cryptocurrency private keys, or deploy sophisticated keyloggers[4]. The sudden appearance of dynamic script injection functions, such as document.createElement('script') or chrome.tabs.executeScript, during a version update is a primary indicator of malicious intent[4]. More advanced attacks leverage the Chrome DevTools Protocol via the chrome.debugger API. By attaching to the browser target or specific tabs, a malicious extension can evaluate arbitrary JavaScript to steal plaintext passwords from password managers, modify critical browser security settings, and even manipulate the contents of internal WebUI tabs and TLS error dialogs[35].

## Advanced Evasion and Obfuscation

A significant challenge in building reliable malware corpora is the escalating sophistication of evasion techniques designed to defeat both static analysis tools and manual store vetting processes. The FV8 project, which utilized a modified V8 JavaScript engine to dynamically identify evasive behaviors, analyzed data from both npm packages and browser extensions. In a dataset of confirmed malicious extensions, the tool identified 164 extensions (82% of the targeted subset) employing at least one advanced evasion technique[38].

Threat actors routinely embed environmental fingerprinting logic to determine the context of their execution. Code deltas frequently reveal conditional logic that assesses user-agent strings, screen resolutions, the presence of specific debugging tools, or even checks against a predefined list of other installed extensions before executing the malicious payload[10]. For example, sophisticated spyware campaigns have been observed actively probing the browser to enumerate thousands of specific extension IDs (using techniques like querying chrome-extension:// resources in the DOM or utilizing fetch() requests) to map the user's exact environment before determining if it is safe to exfiltrate data[10].

To survive the initial automated vetting by the Chrome Web Store, malicious updates often

include logic that purposefully delays execution. By utilizing date functions and basic arithmetic, the extension remains completely dormant for a predetermined number of days after installation. By the time the payload finally detonates, the extension has already been approved by reviewers and distributed to hundreds of thousands of endpoints[23]. Furthermore, unlike benign open-source releases, malicious .crx payloads aggressively utilize minification, string encoding algorithms (such as Base64), and highly complex control flows to conceal their true intent. Effective static differential analysis must therefore employ comprehensive de-obfuscation pipelines or AST normalization to accurately identify the meaningful structural differences between a benign version and its malicious successor[20].

## The Paradigm Shift: Manifest V2 to Manifest V3

Any analysis of browser extension corpora must be contextualized within the fundamental architectural shift from Manifest Version 2 (MV2) to Manifest Version 3 (MV3). Initiated by Google to enhance baseline security, privacy, and performance across the ecosystem, MV3 radically alters the availability of powerful APIs[14].

The most profound security enhancement in MV3 is the deprecation of the synchronous webRequestBlocking API, which has been replaced by the declarativeNetRequest (DNR) API. Under the DNR framework, extensions can no longer inspect and execute custom JavaScript logic to modify network traffic in real-time. Instead, extensions must provide the browser engine with a static list of declarative rules governing how traffic should be handled[3]. This architectural change drastically diminishes the viability of traditional ad-injection malware and complex traffic-hijacking campaigns. Additionally, MV3 forces the transition from persistent background pages to ephemeral service workers, limiting the ability of malicious extensions to maintain continuous, unmonitored background execution[14].

Crucially for threat researchers, MV3 strictly prohibits the execution of remotely hosted code. Extensions are no longer permitted to fetch and execute arbitrary JavaScript from external, attacker-controlled servers via eval() or dynamically injected <script> tags[14]. All executable code must be bundled locally within the .crx package itself. While this policy was implemented to prevent "bait-and-switch" attacks where a benign extension later downloads a malicious payload from a C2 server, threat actors are highly adaptable. Proof-of-concept research analyzing a dataset of 517 confirmed malicious extensions revealed that while MV3 effectively neutralizes 87.8% of APIs traditionally associated with malicious behavior, 56% of the examined malicious extensions could still achieve their objectives by pivoting their attack vectors to remain functional within the MV3 constraints[14].

For intelligence analysts conducting differential analysis, the prohibition of remote hosted code is paradoxically beneficial. Because threat actors can no longer fetch their payloads dynamically at runtime, the entirety of the malicious logic *must* be present within the local .crx file. Consequently, static differential analysis between version pairs—comparing the last benign build against the newly updated package—will yield significantly more comprehensive insights into the malware's mechanics, as the code cannot be hidden on an external server.

| Vulnerability Vector / | Manifest V2 Mechanism | Manifest V3 Mitigation / |
|---|---|---|

| Abuse Type | | Status |
|---|---|---|
| **Network Traffic Interception** | webRequestBlocking (Synchronous JS inspection) | Deprecated; replaced by static declarativeNetRequest rules. |
| **Remote Code Execution** | eval(), injecting external <script src="..."> | Strictly prohibited. All code must be bundled in the .crx. |
| **Persistent Background Execution** | Background Pages | Replaced by ephemeral Service Workers. |
| **DOM Manipulation / Injection** | chrome.tabs.executeScript | Migrated to chrome.scripting.executeScript (Requires explicit permissions). |
| **Cookie Exfiltration** | chrome.cookies | Still vulnerable if the extension is granted permission. |

# Conclusion

The browser extension ecosystem remains a highly volatile and fiercely contested frontier in cybersecurity. As the demarcation between traditional desktop applications and web browsers continues to dissolve, extensions retain the capability to compromise vast repositories of sensitive user data, bypass established network security controls, and fundamentally alter digital reality through seamless DOM manipulation. The threat environment is continually exacerbated by the silent, automatic update mechanisms that empower threat actors to instantaneously convert trusted, benign tools into weaponized assets via supply chain compromises and developer account takeovers.

Defending against these rapidly evolving threats requires moving beyond static, point-in-time analysis. It demands a rigorous, longitudinal approach to threat intelligence, heavily reliant on the proactive acquisition and differential analysis of consecutive extension versions. By leveraging robust public datasets—ranging from automated IOC feeds like chrome-mal-ids and malicious_extension_sentry to massive, historically contextualized academic corpora like the "You've Changed" dataset—researchers can pinpoint the precise moment and mechanism of compromise. Furthermore, by mastering the mechanics of first-party Omaha update protocols, third-party historical archives, and open-source developer ecosystems, security

teams can proactively construct the version archives necessary to trace the execution flow of emerging threats.

As the browser ecosystem transitions fully to Manifest V3, the tactics employed by malicious actors will inevitably shift to circumvent new restrictions. However, because MV3 forces all executable code to reside locally within the extension package, the continuous, automated extraction and structural analysis of update deltas will only increase in efficacy, solidifying differential analysis as the most critical methodology in the modern browser security arsenal.

**Works cited**


1. UNPACKING DATA EXFILTRATION AND MALICIOUS BEHAVIOURS, https://arxiv.org/pdf/2512.10029
2. Mozilla Addons - Archiveteam, https://wiki.archiveteam.org/index.php/Mozilla_Addons
3. Towards Browser Controls to Protect Cookies from Malicious ... - arXiv, https://arxiv.org/html/2405.06830v3
4. Browser Extension Security | Pentest Book, https://www.pentest-book.com/others/browser-extension-security
5. Hulk: Eliciting Malicious Behavior in Browser Extensions, https://www.semanticscholar.org/paper/Hulk%3A-Eliciting-Malicious-Behavior-in-Browser-Kapravelos-Grier/604c12c59d6a2af0cfda6c09ddfcc718d54a470f
6. Detecting Malicious Browser Extensions through their Update Deltas, http://www.mathcs.richmond.edu/~dszajda/classes/cs334/Spring_2021/papers/pantelaios_Detecting_Malicious_Browser_Extensions.pdf
7. GitHub - The-Privacy-Commons-Institute/chrome-mal-ids: Effort to, https://github.com/mallorybowes/chrome-mal-ids
8. Towards Browser Controls to Protect Cookies from Malicious ... - arXiv, https://arxiv.org/html/2405.06830v1
9. toborrm9/malicious_extension_sentry: Malicious Extension Database, https://github.com/toborrm9/malicious_extension_sentry
10. LinkedIn extension-fingerprinting analysis - GitHub, https://github.com/jaylane/linkedin-spyware-analysis
11. toborrm9/malicious_extension_sentry: Malicious Extension Database, https://b.hatena.ne.jp/entry/s/github.com/toborrm9/malicious_extension_sentry
12. LICENSE.md - toborrm9/malicious_extension_sentry · GitHub, https://github.com/toborrm9/malicious_extension_sentry/blob/main/LICENSE.md
13. Towards Browser Controls to Protect Cookies from Malicious ... - arXiv, https://arxiv.org/html/2405.06830v2
14. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions, https://arxiv.org/html/2404.08310v1
15. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions, https://madweb.work/papers/2024/madweb24-pantelaios.pdf
16. Hulk: Eliciting Malicious Behavior in Browser Extensions, https://www.researchgate.net/publication/264991585_Hulk_Eliciting_Malicious_Behavior_in_Browser_Extensions

17. Hulk: Eliciting Malicious Behavior in Browser Extensions - ICIR, https://www.icir.org/vern/papers/hulk-usesec14.pdf
18. Provenance: privacy:browser_extensions - Measure The Web, https://measuretheweb.org/provenance/privacy/browser_extensions
19. Did I Vet You Before? Assessing the Chrome Web Store ... - arXiv, https://arxiv.org/html/2406.00374v2
20. (PDF) Malicious GenAI Chrome Extensions: Unpacking Data, https://www.researchgate.net/publication/398602720_Malicious_GenAI_Chrome_Extensions_Unpacking_Data_Exfiltration_and_Malicious_Behaviours
21. Tuhinshubhra/ExtAnalysis: Browser Extension Analysis Framework, https://github.com/Tuhinshubhra/ExtAnalysis
22. GitHub - chrrel/extensions: A web scanning tool for analysing, https://github.com/chrrel/extensions
23. Malicious Extensions - What They Are And How To Fight Them, https://profero.io/blog/malicious-extensions-what-they-are-and-how-to-fight-them/
24. Installation and Configuration Guide - Oracle Help Center, https://docs.oracle.com/en/applications/enterprise-performance-management/smart-view/22.100/incsv/GUID-5211964F-04CB-40C3-BF66-EA9CBEC53FF7.pdf
25. Google Chrome for Business: Configuration and Deployment, https://docs.google.com/a/google.com/document/d/12my4PJkmCDbRjoS5Ske2DI0bejwwEV_GDnf4bjyVWMQ/edit
26. Use alternative installation methods | Chrome Extensions, https://developer.chrome.com/docs/extensions/how-to/distribute/install-extensions
27. Domains to allow for Firefox - Mozilla Support, https://support.mozilla.org/en-US/kb/domains-allow-firefox
28. Clay Crx File 0.71.0 for Chrome (Old Version) - Crx4Chrome.com, https://www.crx4chrome.com/crx/310462/
29. Error: Manifest version 2 is deprecated, and support will be removed, https://superuser.com/questions/1783948/error-manifest-version-2-is-deprecated-and-support-will-be-removed-in-2023-b
30. Clay Crx File 0.75.0 for Chrome (Old Version) - Crx4Chrome.com, https://www.crx4chrome.com/crx/407626/
31. Chrome Extension & Mobile App Analytics — Chrome-Stats, https://chrome-stats.com/
32. Download CRX Emulator | Free download - Chrome-Stats, https://chrome-stats.com/d/bekkpoinfafbjglppgdobfdeckghdhlo/download
33. Download LINE | Free download - Chrome-Stats, https://chrome-stats.com/d/ophjlpahpchlmihnnnihgmmeilfjmjjc/download
34. CRX Downloader - Download Chrome & Firefox Extensions, https://chrome-stats.com/d/cdbblkdjkdlpgpcelfmdjgbmkcajolje
35. Abusing the Chrome DevTools Protocol through the Debugger API, https://arxiv.org/html/2305.11506v2
36. In-Browser Detection of Malicious Third-Party Content Inclusions,

https://www.ccs.neu.edu/home/mkharraz/publications/Inclusion-FC.pdf
37. Abusing the Chrome DevTools Protocol through the Debugger API, https://www.researchgate.net/publication/370938142_Chrowned_by_an_Extension_Abusing_the_Chrome_DevTools_Protocol_through_the_Debugger_API
38. FV8: A Forced Execution JavaScript Engine for Detecting Evasive, https://arxiv.org/html/2405.13175v1
39. Acquiring Malicious Browser Extension Samples on a Shoestring, https://pberba.github.io/crypto/2024/09/14/malicious-browser-extension-genesis-market/